A Proof-Producing Base-19 Obstruction in a Reverse-and-Add Constraint System

8128

August 2026

Abstract

We study one frozen, finite constraint system extracted from a six-phase reverse-and-add topology in base 19. Exact radial-frontier replay ends in the empty set for a 409-equation candidate and for a 408-equation subset. A complete one-equation-deletion audit makes the latter computationally deletion-minimal. Lean separately proves the generic implication from a closed empty frontier to unsatisfiability; the concrete frontier replays remain computational claims. For the exact 4914-bit packed encoding of the 408-equation core, Lean also proves `Base19Radial.packedMinimalCoreUnsat` through `bv_decide` and LRAT certificate checking. That concrete theorem uses a disclosed native certificate-evaluation axiom, and the integer-to-packed correspondence remains a paper proof rather than a formal Lean bridge. This bounded obstruction is not a theorem about the unbounded reverse-and-add process and does not establish a base-19 Lychrel number.

Contents

1 Orientation **2**

2 The frozen base-19 system **2**

3 Radial decomposition **3**

4 Deletion-minimal obstruction **5**

5 Packed finite encoding **5**

6 Proof-producing Lean/LRAT check **7**

7 Assurance ladder **8**

8 Reproducibility and limitations **8**

9 Related work and conclusion **9**

A Reproducibility record **9**

A.1 Immutable identifiers 9

A.2 Data generation and Lean check 10

A.3 Pinned offline PDF build 10

1 Orientation

For a base-b numeral, a *reverse-and-add step* adds the represented integer to the integer represented by the reversed digit word. Reversal couples digits at opposite ends, whereas carrying couples adjacent columns. The elementary operation therefore becomes a globally coupled system when several steps are required to have a prescribed repeating shape.

Example 1.1 (A decimal step)**.** *Take the two-digit decimal word* 47*. Reversing and adding gives*

$$\begin{array}{r} 47 \\ +74 \\ \hline 121 \end{array} \qquad \textit{since} \qquad \begin{aligned} 7+4+0 &= 1+10\cdot 1,\\ 4+7+1 &= 2+10\cdot 1,\\ 0+0+1 &= 1+10\cdot 0. \end{aligned}$$

The three equalities are the units, tens, and new leading column. They also show why zero boundary atoms and explicit carry variables are convenient: the same equation describes every column.

The topology studied here was prompted by David Seal's reported period-six, self-similar base-17 construction, preserved in Brown's account [Bro96]. OEIS A060382 records conjectural least starts of palindrome-free reverse-and-add sequences by base and cautions that only its base-two entry is proved [OEI26]. These sources provide motivation and provenance only. Neither supplies a premise for the finite base-19 calculation below, and no claim of novelty or resolution of the OEIS entry is made.

Key idea

The question answered here is deliberately finite: does one explicitly listed family of bounded digit-and-carry equations have a solution? Opposite columns are processed together, leaving only a small radial frontier between the processed exterior and the unprocessed center.

2 The frozen base-19 system

Set $b = 19$. Every digit variable has domain $D = \{0, 1, \dots, 18\}$, every carry variable has domain $K = \{0, 1\}$, and the leading digit of each phase is restricted to $D \setminus \{0\}$. Literal atoms are fixed integers, not variables. At phase p and column k, write $\ell_{p,k}$ and $r_{p,k}$ for the source atoms met from the two ends, $t_{p,k}$ for the target atom, and $c_{p,k-1}, c_{p,k}$ for the incoming and outgoing carries. A missing end atom or boundary carry is the literal zero. The exact column constraint is

$$\ell_{p,k} + r_{p,k} + c_{p,k-1} = t_{p,k} + 19c_{p,k}. \tag{1}$$

Thus every symbol and every domain in Eq. (1) is an ordinary bounded integer; modular arithmetic enters only in the later packed encoding.

There are six phases $p \in \{0, 1, \dots, 5\}$. Each phase word has three variable blocks. Their lengths are

$$(26, 25, 31) \quad \text{for } p = 0, \qquad (29, 23, 31) \quad \text{for } p = 1, \dots, 5.$$

For $n \geq 0$, insert a constant run of length n between consecutive blocks. The two inserted digits are $(0, 18)$ for phases $0, 2, 3, 4, 5$ and $(18, 0)$ for phase 1. Denote the resulting word by $W_p(n)$.

Words are written most-significant digit first. More formally, let $s_p = |W_p(0)|$, set $V_p = W_{p+1}(0)$ when $0 \leq p < 5$ and $V_5 = W_0(1)$, and let $m_p = |V_p|$. Write these source and target words as

$$W_p(0) = x_{p,0}x_{p,1}\cdots x_{p,s_p-1}, \qquad V_p = y_{p,0}y_{p,1}\cdots y_{p,m_p-1}.$$

The source and target lengths are $s_0 = 82$, $s_p = 83$ for $1 \le p \le 5$, $m_p = 83$ for $0 \le p < 5$, and $m_5 = 84$. Arithmetic column $k = 0$ is the least-significant target column. For $0 \le k < m_p$, the written-index map is

$$\ell_{p,k} = \begin{cases} x_{p,k}, & k < s_p, \\ 0, & k \ge s_p, \end{cases} \qquad r_{p,k} = \begin{cases} x_{p,s_p-1-k}, & k < s_p, \\ 0, & k \ge s_p, \end{cases} \tag{2}$$

$$t_{p,k} = y_{p,m_p-1-k}. \tag{3}$$

In particular, $\ell_{p,k}$ advances from the written left (most-significant) end, while $r_{p,k}$ advances from the written right (least-significant) end; missing source positions are literal zero. The initial and final boundary carries are zero:

$$c_{p,-1} = 0, \qquad c_{p,m_p-1} = 0. \tag{4}$$

For $p < 5$, phase p therefore imposes the column equations taking $W_p(0)$ to $W_{p+1}(0)$. Phase 5 takes $W_5(0)$ to $W_0(1)$; this is the single growth step that closes the six-phase cycle. In particular, phase-5 column 83 has target index $m_5 - 1 - 83 = 0$. Since $83 = s_5$, both source atoms are literal zero, the target is the leading atom `p0:f0:d0`, and the final outgoing carry is $c_{5,83} = 0$. Its retained equation is therefore

$$0 + 0 + c_{5,82} = \texttt{p0:f0:d0} + 19 \cdot 0. \tag{5}$$

The omitted phase-5 column 82 removes that column equation, but its named outgoing carry $c_{5,82}$ remains the incoming carry in Eq. (5).

Only two exterior coordinate bands from each full addition are retained. They are

$$\begin{aligned} p = 0: &\quad \text{0–33 and 48–82}, \\ p = 1, 2, 3, 4: &\quad \text{0–33 and 49–82}, \\ p = 5: &\quad \text{0–33 and 49–83}. \end{aligned}$$

The builder then omits `phase=5,n=0,column=82`. The remaining equations, in their canonical manifest order, form the frozen candidate C. It has exactly 409 equations over 409 active digit variables and 410 active carry variables. Each equation carries a provenance label of the form `phase=p,n=0,column=k`; the manifest also fixes every literal, variable name, and carry endpoint.

Finally let

$$e_\star = \texttt{phase=0,n=0,column=82}, \qquad C^\circ = C \setminus \{e_\star\}.$$

Consequently C° contains 408 equations. This removal must not be confused with the earlier phase-5 omission used to define the candidate itself. The three objects used below are therefore: the 409-equation candidate C, its 408-equation subset C°, and the complete audit of every single-equation deletion of C.

3 Radial decomposition

The retained columns are paired from the outside toward the center across all six phases. The promoted certificate stores 35 atomic layers, labeled $-1, 0, \ldots, 33$. The label -1 is a one-equation bootstrap layer. For cleaner notation here, merge that bootstrap with stored layer 0 and call the result L_0; for $1 \le i \le 33$, let L_i be stored layer i. This harmless coarsening gives the 34 transfers shown in Fig. 1.

For a set of equations E, let $\mathrm{Var}(E)$ be its variables. After processing through L_i, the live interface is the canonically sorted list

$$I_i = \mathrm{Var}(L_0 \cup \cdots \cup L_i) \cap \mathrm{Var}(L_{i+1} \cup \cdots \cup L_{33}), \qquad I_{-1} = (). \tag{6}$$

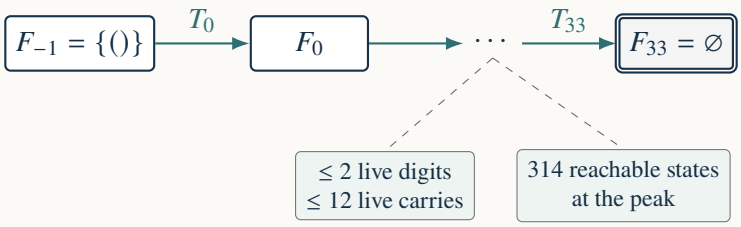


Figure 1: The radial frontier computation for C°. The double outline, not color alone, marks the empty terminal frontier. The callouts are generated from the frozen core certificate; the drawing is schematic and is not itself proof evidence.

A state on I_i is a domain-respecting tuple in that fixed order. Put $H_i = \mathrm{Var}(L_i) \setminus (I_{i-1} \cup I_i)$. For a state s on I_{i-1}, the exact local transfer is

$$T_i(s) = \left\{ u \text{ on } I_i : \begin{array}{l} \text{there is a domain-respecting assignment } h \text{ on } H_i, \\ s, u, h \text{ agree on overlaps and together satisfy every equation in } L_i \end{array} \right\}. \tag{7}$$

Starting from the one empty tuple, define the reachable frontiers by

$$F_{-1} = \{()\}, \qquad F_i = \bigcup_{s \in F_{i-1}} T_i(s) \quad (0 \le i \le 33). \tag{8}$$

This is exact variable elimination: variables that do not cross the cut are composed locally and then forgotten. Its usefulness comes from the bounded live interface, the same general phenomenon behind path-structured and bounded-width constraint algorithms [Fre90, DP89]. Those references provide methodological context only; they do not verify a transition or a numerical claim in this artifact.

For C°, the independently replayed transfers satisfy $F_{33} = \varnothing$. At every stored cut there are at most 2 live digit variables and 12 live carry variables, and the largest serialized reachable frontier contains 314 states. These values are loaded from `generated-results.tex`, whose renderer checks the frozen certificate rather than duplicating them as manuscript literals.

Exact equality in Eq. (8) is more than soundness needs. Call a sequence $G_{-1}, G_0, \ldots, G_{33}$ *closed* when $() \in G_{-1}$ and

$$s \in G_{i-1} \text{ and } u \in T_i(s) \quad \Longrightarrow \quad u \in G_i \qquad (0 \le i \le 33).$$

Theorem 3.1 (Closed empty frontier)**.** *Suppose the layers cover every equation of a bounded constraint problem and their adjacent interfaces agree. If $G_{-1}, \ldots, G_{33}$ is closed and $G_{33} = \varnothing$, then the problem has no satisfying assignment.*

Proof. If a were a global satisfying assignment, its restriction to the empty initial interface would lie in G_{-1}. Assume inductively that $a|_{I_{i-1}} \in G_{i-1}$. The restriction of a to the variables of L_i witnesses $a|_{I_i} \in T_i(a|_{I_{i-1}})$; closure therefore gives $a|_{I_i} \in G_i$. At $i = 33$ this puts a state in the empty set, a contradiction. □

Lean proves precisely this generic implication as `Base19Radial.unsat_of_closed_empty_frontier`: from layer coverage, an aligned closed chain, an initial state, and an empty final frontier, it concludes $\neg\exists a$, `Problem.Satisfies` a. This theorem does not by itself check the concrete serialized transitions for either C or C°. Nor is there presently a Lean theorem bridging `Problem.Satisfies` to the separate packed proposition discussed later; that integer-to-packed correspondence remains a paper-level boundary.

4 Deletion-minimal obstruction

Three computations must be kept distinct.

1. The promoted frontier for the full candidate C covers all 409 equations and ends empty.
2. Removing $e_\star$ = `phase=0,n=0,column=82` gives C°. Its separate promoted frontier covers 408 equations and also ends empty after exact replay.
3. The deletion audit has 409 records, one for each equation $e \in C$. It records 408 complete satisfying assignments for $C \setminus \{e\}$ and 1 UNSAT outcome, occurring exactly when $e = e_\star$.

For every $e \in C^\circ$, the audit's assignment for $C \setminus \{e\}$ also satisfies the smaller system $C^\circ \setminus \{e\}$. The audit replays each stored digit and carry bound, leading-digit condition, and retained integer equality. Combined with the separate empty-frontier replay for C°, this establishes the following carefully qualified result.

Proposition 4.1 (Computational deletion minimality)**.** *Under the correctness of the frozen manifest parser, the bounded-integer encoding, and the exact solver answers replayed by the independent checker, C° is unsatisfiable and $C^\circ \setminus \{e\}$ is satisfiable for every $e \in C^\circ$. Thus C° is deletion-minimal within the frozen candidate.*

The qualifier "computational" is essential here. The 409-equation empty frontier, the 408-equation core replay, and the 409-way deletion audit are frozen and independently rechecked computational artifacts. The generic Lean theorem in Theorem 3.1 validates the implication from suitable closure hypotheses, but this section does not claim that Lean discharges every concrete frontier transition or all deletion witnesses.

What this does not prove

Deletion-minimal means that no single remaining equation can be removed from this core while preserving unsatisfiability. It is *not minimum-cardinality*: a different, smaller unsatisfiable subset has not been excluded. The result gives no uniqueness or all-base theorem, says nothing about the unbounded reverse-and-add process, and does not establish a base-19 Lychrel number.

5 Packed finite encoding

The generated check replaces a tuple of small integers by one bit-vector. The active variables of C° occur in the generator's fixed manifest order: first the 409 digit variables, then the 410 carry variables. Thus there are

$$409 + 410 = 819$$

slots. Each slot has width 6, so the complete assignment is a word

$$A \in \mathrm{BitVec}(819 \cdot 6) = \mathrm{BitVec}(4914).$$

For $0 \le j < 819$, slot j is the unsigned six-bit value extracted from the bit interval beginning at least-significant-bit offset $6j$. In the generated Lean source this operation is `assignment.extractLsb' (index * 6) 6`. The 819 intervals are disjoint and exhaust all 4914 bits; there are no padding bits.

Write $s_j(A) \in \{0, \dots, 63\}$ for the ordinary representative of the jth slot. Let $\mathsf{PackedCore}_{408}(A)$ be the conjunction emitted by the deterministic renderer. Its 1233 conjuncts divide into four audited categories:

Constraint category	Count
Unsigned digit bounds $s_j(A) < 19$	409
Unsigned carry bounds $s_j(A) < 2$	410
Nonzero leading digits $s_j(A) \neq 0$	6
Retained column equations	408
Total	$409 + 410 + 6 + 408 = 1233$

The six extra inequalities select the variables `p0:f0:d0`,...,`p5:f0:d0`; each already has a digit-bound conjunct. Literal atoms and boundary carries remain six-bit constants rather than occupying slots. Every retained instance of Eq. (1) is translated literally, with six-bit addition and multiplication, and hence is initially an equality modulo 64.

Proposition 5.1 (Paper equivalence; unformalized bridge)**.** *The bounded integer assignments satisfying C° are in bijection with the words A : BitVec(4914) satisfying* $\mathsf{PackedCore}_{408}(A)$.

Proof. This is a paper proof, not a theorem currently present in Lean. Consider one column and take the ordinary representatives of its five atoms. The bounds give

$$0 \le \ell, r, t \le 18, \qquad c_{\text{in}}, c_{\text{out}} \in \{0, 1\}.$$

Consequently

$$0 \le \ell + r + c_{\text{in}} \le 18 + 18 + 1 = 37, \qquad 0 \le t + 19c_{\text{out}} \le 18 + 19 = 37.$$

Both sides therefore lie in $[0, 37]$. Since `37<64`, two such representatives are equal in $\mathbb{Z}/64\mathbb{Z}$ exactly when they are equal as ordinary integers: their difference is a multiple of 64 with absolute value strictly below 64, and hence is zero. This proves the slotwise equivalence of each bit-vector column equation and Eq. (1); it also shows explicitly that no six-bit operation in a satisfied column can wrap.

For completeness, let $(v_0, \ldots, v_{818})$ be a tuple of slot values in $\{0, \ldots, 63\}$ and pack it as the 4914-bit word represented by

$$\text{pack}(v) = \sum_{j=0}^{818} v_j 2^{6j}.$$

Six-bit extraction gives $s_j(\text{pack}(v)) = v_j$. Conversely, because the extracted blocks partition the entire word, $\text{pack}(s_0(A), \ldots, s_{818}(A)) = A$ for every A. Thus packing and unpacking are inverse operations. An integer model supplies slot values in the required ranges, its six leading digits remain nonzero, and its integer equations imply the modular ones. In the other direction, the unsigned slot bounds recover digits in $\{0, \ldots, 18\}$ and carries in $\{0, 1\}$; the nonzero clauses recover the six leading conditions; and the no-wrap argument recovers every integer equation. The two constructions therefore restrict to the claimed bijection of satisfying assignments. □

The renderer constructs and audits this correspondence against the frozen manifest and records its digest as `18ca75da64c84859d2373ae3e61da81e3ac75a68a33af448b57220e23dfbfa9e`. The proposition above is nevertheless an *unformalized bridge*: no existing Lean declaration maps the earlier `Problem.Satisfies` predicate to $\mathsf{PackedCore}_{408}$. The generic frontier theorem, this generated and audited bridge, and the packed theorem below are separate statements.

6 Proof-producing Lean/LRAT check

Abbreviating the generated conjunction just defined, the exact conclusion of the generated Lean declaration is as follows.

Lean theorem: packed core UNSAT

`Base19Radial.packedMinimalCoreUnsat` has type

$$\forall A : \mathrm{BitVec}(4914), \qquad \neg \mathsf{PackedCore}_{408}(A).$$

Equivalently, no 4914-bit word satisfies the exact inline conjunction of 1233 generated constraints.

At the frozen Lean 4.32.2 source commit, `bv_decide` reflects and bit-blasts the proposition, selects an adjacent `cadical` executable by default, and invokes it with `--lrat`. In the pinned toolchain, `lake env cadical --version` prints `2.1.2`; hence CaDiCaL 2.1.2, not Z3, searches and Lean checks the returned LRAT certificate. The pinned output, rather than the release page alone, identifies the binary [Lea26, CFHH+17, Bc24].

There is a second, essential layer to the checking story. Lean's checker has a proved soundness theorem, but the frozen implementation's `LratCert.toReflectionProof` uses `nativeEqTrue` to evaluate the checker result. That evaluation contributes an additional dependency to the accepted theorem term. The artifact's `#print axioms` output reports the ordinary axiom dependencies

`propext`, `Classical.choice`, `Quot.sound`,

and the artifact-specific native certificate-evaluation axiom

`packedMinimalCoreUnsat._native.bv_decide.ax_1_5`.

Accordingly, the concrete certificate evaluation trusts Lean's native compiler, code generator, and runtime in addition to the proved checker and kernel. The result is proof-producing and substantially narrows the trusted solver boundary, but it is *not kernel-reduction-only*, not fully axiom-free, and not an end-to-end formalization of the bounded integer model.

One recorded development-machine run of the generated module took 51.01 seconds and reached 1.28 GB maximum resident set size. These are measurements of that one run, not a performance guarantee. Regeneration compares the source byte-for-byte, checks the digest binding against the frozen equation manifest, and reruns the declaration under Lean 4.32.2. The exact digest is supplied by the generated results file rather than manually transcribed.

Trust boundary

Four boundaries must not be collapsed. Lean proves the generic theorem that a suitably closed empty frontier implies `Problem.Satisfies` is impossible. The concrete frontier files are instead replayed by exact computation. The frozen integer-to-packed translation is generated and audited but remains the unformalized bridge proved only on paper in Proposition 5.1. Finally, `Base19Radial.packedMinimalCoreUnsat` proves UNSAT for the exact packed proposition, with the native evaluation dependency listed above. Therefore the paper does not claim a single end-to-end Lean theorem from the frozen integer problem to the packed conclusion.

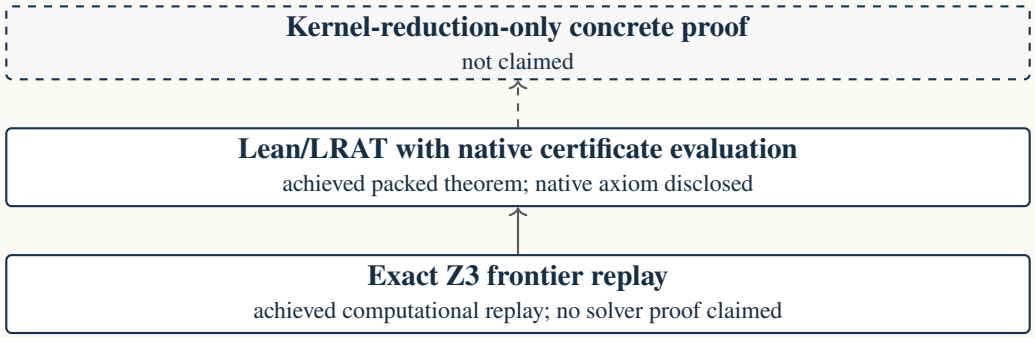


Figure 2: Assurance ladder for the concrete checks. Solid outlines mark achieved levels; the dashed outline marks the unclaimed endpoint. Height indicates assurance scope, not mathematical generality. The diagram is explanatory and is not itself proof evidence.

7 Assurance ladder

The preceding distinctions can be arranged by the scope of their trusted base. The bottom level re-executes the concrete frontier computation using Z3's mathematical integers and exact finite comparisons [Mic26]; it does not attribute a proof object to Z3. The middle level is the achieved packed theorem whose CaDiCaL 2.1.2 LRAT certificate is checked in Lean with native evaluation. A strict concrete proof checked only by kernel reduction would be a further endpoint, not a property of the present artifact.

In particular, moving upward in Fig. 2 must not be read as enlarging the theorem's domain. The frontier replay concerns the frozen bounded integer instance, while the Lean result concerns its separately generated packed proposition. Their correspondence is supplied by the paper proof and audited renderer, not by a hidden claim that the packed theorem also formalizes the frontier certificates.

8 Reproducibility and limitations

The evidence package follows the disclosure principle that computer-aided arguments should identify the role of computation and make the supporting code and data available for scrutiny [Lon24]. Here the roles are separated deliberately. Python regenerates the finite instance, replays the exact frontiers and satisfying assignments, and audits the packed translation. Lean 4.32.2 checks the generic frontier implication, one concrete satisfying neighbor, and the exact packed UNSAT proposition. The generated values used in the paper are loaded from `preprint-v2/generated-results.tex`; its renderer rejects drift in the promoted hashes and in the generated packed source.

The compact command record in Section A gives two routes. The first regenerates both generated files and directly runs Lean on the packed theorem. The second builds this PDF in the digest-pinned TeX Live image with an AMD64 platform and networking disabled. The artifact-level commands `./publication/base19-radial/reproduce.sh quick` and `... reproduce.sh full` respectively perform the promoted replay and the substantially longer complete regeneration; the repository README describes their scope and prerequisites.

The reported 51.01-second time and 1.28 GB maximum resident set size describe one recorded run on the development machine. They are not bounds, benchmarks, or promises for another host. The full LRAT certificate and the generated 4914-bit proposition are intentionally not printed here: both are machine-oriented evidence, while their identities, generator, checker route, theorem name, and relevant hashes are recorded below.

The limitations are logical as well as practical. Concrete frontier closure and the full deletion audit

are exact computational replays, not individual Lean proof terms. Proposition 5.1 is the unformalized integer-to-packed bridge. The packed theorem contains the native certificate-evaluation axiom shown in Section 6. Deletion minimality is relative to one frozen candidate and does not imply minimum cardinality, uniqueness, or any statement about indefinite iteration.

9 Related work and conclusion

Seal's reported base-17 construction and OEIS A060382 place this calculation within the historical study of reverse-and-add behavior [Bro96, OEI26]. Brown's page is a transcription, and the OEIS entry itself labels the general sequence conjecturally and says only the base-two term is proved. They are therefore provenance and motivation, never evidence for an equation, certificate, or theorem in this artifact.

The radial computation belongs methodologically to bounded-interface constraint processing. Freuder's width analysis exhibits the familiar linear dependence on problem size with exponential dependence on fixed width, while Dechter and Pearl describe clustering a constraint network into an acyclic structure for backtrack-free processing [Fre90, DP89]. The present layers and transitions are defined directly and replayed from the frozen instance; those general results explain the tractability mechanism but do not certify this computation.

The two computational routes also use different solver interfaces. Z3's integer sort has mathematical-integer semantics, and its linear-integer arithmetic machinery uses cuts and branch-and-bound [Mic26]; this route supplies exact replay but no solver proof object here. In the SAT certificate lineage, DRAT-trim checks and trims expressive clausal proofs [WHH14]; LRAT adds explicit hints intended to support a small efficient certified checker [CFHH+17]; and FRAT lets a solver communicate additional information to a later elaborator [BCH22]. The achieved packed route is specifically CaDiCaL 2.1.2 certificate search followed by Lean's LRAT checker, not a claim that this artifact emits every one of those formats [Bc24].

Lean combines an interactive theorem prover with an extensible functional programming language [dMU21]. At the frozen Lean 4.32.2 commit, its `bv_decide` implementation reflects and bit-blasts the finite proposition, processes the LRAT result with a proved-sound checker, and uses native evaluation to construct the accepted equality proof [Lea26]. This explains both the gain over accepting a bare UNSAT answer and the explicitly larger trust boundary of the concrete theorem.

In conclusion, exact replay establishes that the frozen 408-equation bounded-integer core is computationally deletion-minimal within its 409-equation candidate. For the separately generated exact six-bit encoding, Lean proves `Base19Radial.packedMinimalCoreUnsat`, namely that no 4914-bit assignment satisfies its 1233 constraints. The final bounded result is therefore the UNSAT theorem for that exact packed proposition, qualified by the paper-level integer-to-packed bridge and by the disclosed native certificate-evaluation axiom; it is not a kernel-reduction-only theorem.

A Reproducibility record

A.1 Immutable identifiers

The three Lean declarations used in the assurance discussion are:

- `Base19Radial.unsat_of_closed_empty_frontier` (generic frontier soundness);
- `Base19Radial.deleteP0C0Sat` (one promoted satisfying neighbor); and
- `Base19Radial.packedMinimalCoreUnsat` (UNSAT for the packed core).

The checked satisfying neighbor removes

`phase=0,n=0,column=0`.

The frozen equation-manifest SHA-256 is

`18ca75da64c84859d2373ae3e61da81e3ac75a68a33af448b57220e23dfbfa9e`.

The promoted candidate-frontier, core-frontier, and complete deletion-audit SHA-256 values are, respectively,

`1ce1b3c59bd95264e5338f7fba5bdffa15fb7635917964980057615395cf138c`,
`7635e7188197021cfc15cbad6d087f6fdbfa7a8bbfe4e4ed9b91a15a89498c67`,
`0566f6e2025aec28fe0d2f824f1f6b343cf912e9191a9036629a2a1d4ef03325`.

They identify:

- `certificates/frontier.json`;
- `certificates/minimal-core-frontier.json`; and
- `certificates/deletions.jsonl`.

The authoritative inventory is

`publication/base19-radial/SHA256SUMS`.

A.2 Data generation and Lean check

From the repository root, regenerate the manuscript macros and the packed Lean source, then check the packed theorem under the toolchain pinned to Lean 4.32.2:

```
uv run --frozen python \
  publication/base19-radial/scripts/render_preprint_v2_data.py \
  --artifact-dir publication/base19-radial \
  --output publication/base19-radial/preprint-v2/generated-results.tex
uv run --frozen python \
  publication/base19-radial/scripts/render_lean_bv_core.py \
  publication/base19-radial/instance/equations.json \
  publication/base19-radial/lean/Base19Radial/Generated/CoreBV.lean
(cd publication/base19-radial/lean &&
  lake env cadical --version &&
  lake env lean Base19Radial/Generated/CoreBV.lean)
```

The version command prints `2.1.2`, and the final command prints the exact native axiom dependency discussed in Section 6. The subshell returns to the repository root. Regeneration emits the full 4914-bit conjunction to `CoreBV.lean`; this appendix records the command and identity instead of reproducing that source or its LRAT certificate.

A.3 Pinned offline PDF build

The Dockerfile pins the TeX Live image

`ghcr.io/xu-cheng/texlive-full`

at the digest

`sha256:1a2027fd03aa64455cf36c927c1dfccffc8fc8a3cb11012333be815f8b33e956`.

Run the Docker commands from the repository root. After that image is present locally, the exact AMD64, network-disabled build is:

```
docker build --platform linux/amd64 --network none \
  -f publication/base19-radial/Dockerfile.paper \
  -t base19-radial-paper publication/base19-radial
docker run --rm --platform linux/amd64 --network none \
  -v "$PWD/publication/base19-radial/preprint-v2:/paper" \
  base19-radial-paper
```

The container invokes `build.sh`, fixes the build epoch and timezone, and rejects unresolved citations, unresolved references, overfull boxes, or an empty PDF.

References

[Bc24] Armin Biere and contributors. CaDiCaL sat solver, release 2.1.2, 2024. Official rel-2.1.2 release, 19 December 2024, commit 3ff42f0; accessed 9 August 2026.

[BCH22] Seulkee Baek, Mario Carneiro, and Marijn J. H. Heule. A flexible proof format for SAT solver-elaborator communication. *Logical Methods in Computer Science*, 18(2):3:1–3:21, April 2022.

[Bro96] Kevin S. Brown. Digit reversal sums leading to palindromes, 1996. Transcribes David Seal's January 1996 report; accessed 8 August 2026.

[CFHH+17] Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Matt Kaufmann, and Peter Schneider-Kamp. Efficient certified RAT verification. In *Automated Deduction – CADE 26*, volume 10395 of *Lecture Notes in Computer Science*, pages 220–236. Springer, 2017.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[DP89] Rina Dechter and Judea Pearl. Tree clustering for constraint networks. *Artificial Intelligence*, 38(3):353–366, April 1989.

[Fre90] Eugene C. Freuder. Complexity of K-tree structured constraint satisfaction problems. In *Proceedings of the Eighth National Conference on Artificial Intelligence*, pages 4–9. AAAI Press, 1990.

[Lea26] Lean FRO and contributors. Lean 4.32.2: Proof-producing bit-vector decision procedure and LRAT checker. Lean 4 source tree, commit f3b06c705e6c85f5314019d5d3baab0fec5b580c, 2026. Accessed 8 August 2026.

[Lon24] London Mathematical Society. Computer aided proofs, November 2024. Policy page, accessed 8 August 2026.

[Mic26] Microsoft Research. Z3 guide: Arithmetic, 2026. Accessed 8 August 2026.

[OEI26] OEIS Foundation Inc. The on-line encyclopedia of integer sequences, a060382, 2026. Accessed 8 August 2026.

[WHH14] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014.